

G

GQG System

Modulo de Credito - Cuentas a Cobrar (Contado / Credito Regular / Credito Irregular)

Documento de Defensa - Primer Examen Final

Materia: Ingenieria de Software III - Enfasis en ASI

Unidad 3: Mediciones y Tecnicas de Estimacion

Facultad Politecnica - Universidad Nacional de Asuncion

Este documento responde, criterio por criterio, la rubrica de la Guia de actividad del Primer Examen Final. El prototipo esta implementado y desplegado en linea (<https://gqg.ianmrc.dev>): app full-stack Next.js 15 + MySQL/MariaDB, con el trigger de generacion de cuotas como nucleo de la logica de negocio.

Alcance: el requerimiento habla de compras y ventas

Si, es correcto. El requerimiento del cliente dice textualmente que el sistema 'cuenta con los procesos de compras y ventas, los cuales solo permiten facturaciones de contado', y pide permitir indicar 'si el proceso de compra o venta se realizara' en una de las modalidades.

Sin embargo, el punto 3 del entregable acota el alcance del prototipo de forma explicita:

```
"Ensayar el codigo del trigger de la base de datos para CUALQUIERA de los procesos (compras o ventas) que implemente esta solucion."
```

Por eso el prototipo implementa el proceso de VENTAS (cuentas a cobrar), que es representativo de toda la logica pedida: eleccion de modalidad, generacion automatica de cuotas y vencimientos. COMPRAS es el proceso espejo (cuentas a pagar) y se resuelve reutilizando exactamente la misma estructura: por eso la tabla CUENTAS_COBRAR incluye las columnas 'tabla' y 'tabloid', que la hacen polimorfica y reusable para registrar tambien cuentas originadas en compras sin cambiar el modelo. Esta decision esta alineada con lo que pide el enunciado y mantiene el prototipo enfocado.

1. Comprension del requerimiento (20 pts)

El problema

GQG System es un sistema de facturacion que hoy solo opera de contado. El negocio crecio, la cartera de clientes aumento y las reglas cambiaron: se necesita poder vender (y comprar) a credito de forma administrativa, eligiendo la modalidad de pago al registrar el comprobante.

Las tres modalidades de pago

- Contado (CO): la cuenta es unica y se cancela en la misma fecha de la factura.
- Credito Regular (CR regular): varias cuotas con vencimiento mensual regular a partir de la fecha de la factura. Ejemplo: CR-30/60/90 dias (3 cuotas a 30, 60 y 90 dias).
- Credito Irregular (CR irregular): varias cuotas, pero los dias de cada vencimiento se indican manualmente. Ejemplo: CR-30/45/60 dias.

Que se implementa y como se justifica la solucion

Todos los datos por encima del detalle de cuotas ya existen en la factura; lo que faltaba implementar es justamente el detalle 'Cuotas' de la cuenta (nro de cuota, importe, vencimiento, cobrado). La solucion propuesta es un trigger en la base de datos que, al registrar la venta, genera automaticamente esas cuotas segun el plazo elegido. Asi se garantiza consistencia (la suma de las cuotas es siempre igual al total de la factura) sin que el usuario las cargue a mano, que es la necesidad concreta del cliente.

2. Diseno del sistema y base de datos (20 pts)

El modelo se agrupa en catalogos, el nucleo de plazos, productos, ventas y cuentas a cobrar.

Tablas y por que existen

- MONEDAS: moneda local y su cantidad de decimales. El campo 'decimales' lo usa el trigger para truncar/redondear el importe de cada cuota (Guarani = 0 decimales).
- TIPOS_DOCUMENTO: Contado/Credito/Nota Credito, con 'tipoid' (0=Contado, 1=Credito) que debe coincidir con el del plazo.
- EMPRESAS, DEPOSITOS, CLIENTES: datos maestros de cabecera de la factura.
- PLAZOS (nucleo del requerimiento): cabecera de la modalidad. Campos clave: 'cuotas' (cantidad) e 'irregular' (0=regular x30, 1=irregular con dias manuales).
- PLAZO_DETALLES: solo para plazos irregulares; guarda los 'dias' de cada cuota desde la fecha de la factura (ej. 30, 45, 60).
- PRODUCTOS / PRODUCTO_DETALLE: catalogo de items, con IVA por producto.
- VENTAS: cabecera del comprobante. Se le agrego 'plazoid' (no estaba en el script de catedra) porque es el dato que el trigger necesita para saber como armar las cuotas.
- VENTA_DETALLES: lineas de la venta (precio, cantidad, IVA, total).
- CUENTAS_COBRAR: lo que genera el trigger (cuota, importe, vence, cobrado). 'tabla'/'tblaid' la hacen reusable para ventas y, a futuro, compras.

Entidades y relaciones clave

- VENTAS N:1 CLIENTES, N:1 PLAZOS, N:1 MONEDAS, N:1 TIPOS_DOCUMENTO, N:1 DEPOSITOS.
- PLAZO_DETALLES N:1 PLAZOS (el detalle de dias pertenece a un plazo irregular).
- VENTA_DETALLES N:1 VENTAS y N:1 PRODUCTO_DETALLE (por codbarra); PRODUCTO_DETALLE N:1 PRODUCTOS.
- CUENTAS_COBRAR se relaciona con VENTAS de forma logica via (tabla='VENTAS', tblaid=VENTAS.id).

El esquema completo (CREATE TABLE con todas las claves foraneas) puede visualizarse como diagrama entidad-relacion importando db/01_schema.sql en Azimutt (azimutt.app).

3. Prototipo de interfaz grafica (15 pts)

El prototipo es una aplicacion web funcional y desplegada, en espanol. Pantallas:

- Login (autenticacion basica por sesion).
- Nueva venta: selector de cliente, fecha, total, toggle CONTADO/CREDITO, selector de plazo (regular o irregular) y vista previa en vivo de las cuotas con su importe y vencimiento.
- Ventas: listado de comprobantes emitidos.
- Cuentas a cobrar: detalle por venta con sus cuotas (cuota, importe, vence, cobrado).
- ABM de Clientes y ABM de Plazos (alta y consulta).
- Factura imprimible (comprobante no fiscal con liquidacion de IVA y plan de cuotas).

Controles para gestionar las modalidades de pago

- Toggle Contado/Credito que filtra los plazos validos para esa modalidad.
- Selector de plazo que distingue regular vs irregular.
- Preview de cuotas que muestra, antes de confirmar, como quedaria el plan (numeradas 01/03, fechas DD/MM/YYYY) y un indicador de que la suma 'cuadra' con el total.

Funcionalidades implicitas (calidad)

- Validaciones: cliente, plazo y total obligatorios; mensajes de error claros.
- Control de secuencia: no se genera la venta si faltan datos.
- Diseno amigable y responsive (uso en tablet en el mostrador).

4. Implementacion tecnica: el trigger (15 pts)

El nucleo de la logica de negocio es el trigger ins_ventas (AFTER INSERT ON VENTAS): al registrar una venta, genera las filas de CUENTAS_COBRAR. Pasos:

- 1) Valida que el tipoid del documento coincida con el tipoid del plazo; si no, aborta con SIGNAL SQLSTATE '45000' (no se puede registrar un credito con un plazo de contado).
- 2) Lee la cantidad de cuotas y si el plazo es irregular.
- 3) Lee los decimales de la moneda.
- 4) Importe base de cada cuota = TRUNCATE(total / cuotas, decimales).
- 5) Ultima cuota = total - base*(cuotas-1): absorbe el redondeo para que la suma sea exacta.
- 6) Inserta las cuentas: si es contado, 1 cuota que vence en la fecha de la factura; si es credito, un bucle por cada cuota con dias = i*30 (regular) o PLAZO_DETALLES.dias (irregular), usando DATE_ADD sobre la fecha de la factura.

Fragmento esencial

```
SET importecuota = TRUNCATE(NEW.totalfactura / nCuotas, decimalesMon);
SET ultimacuota = NEW.totalfactura - importecuota * (nCuotas - 1);
...
IF isIrregular = 1 THEN
    SELECT IFNULL(pd.dias, i*30) INTO dias FROM PLAZO_DETALLES pd
    WHERE pd.plazoid = NEW.plazoid AND pd.cuota = i;
ELSE SET dias = i * 30;
END IF;
INSERT INTO CUENTAS_COBRAR(tabla,tablaid,cuota,importe,cobrado,vence)
VALUES('VENTAS',NEW.id,i,imp,0,DATE_ADD(NEW.fechafactura,INTERVAL dias DAY));
```

Ejemplo verificado (caso del cliente)

Total 584.226 Gs, plazo CR irregular de 3 cuotas (30/45/60 dias). El trigger genera 194.742 + 194.742 + 194.742 = 584.226 (exacto), con vencimientos a 30, 45 y 60 dias de la fecha de la factura. La eleccion 'Opcion A' (division simple, sin interes, ultima cuota absorbe el redondeo) mantiene el importe parejo y la suma cuadrada.

5. Presentacion oral y argumentacion (15 pts)

Guion sugerido para la defensa (de lo general a lo concreto):

- Problema del cliente y las 3 modalidades (criterio 1).
- Modelo de datos: mostrar el diagrama (Azimutt) y justificar PLAZOS, PLAZO_DETALLES, VENTAS.plazoid y CUENTAS_COBRAR.
- Demo en vivo: cargar una venta a credito irregular y mostrar como el trigger arma las cuotas.
- Explicar el trigger linea por linea (validacion, truncado por moneda, ultima cuota, DATE_ADD).
- Cerrar con calidad: validaciones, diseno y despliegue real.

Terminologia a usar: trigger, integridad referencial, regla de negocio, clave foranea, redondeo/truncado, prototipo, requerimiento funcional vs implicito.

6. Trabajo en equipo y coordinacion (10 pts)

- Trabajo sobre repositorio Git compartido del equipo (GQG-System), con cambios via fork + Pull Request para revision.
- Division de responsabilidades: modelo de datos y trigger, interfaz, despliegue/infra y documentacion.
- La defensa se reparte por criterios para que todos los integrantes expongan una parte.

7. Documentacion de apoyo (5 pts)

Entregables de documentacion (en el drive y el repo):

- documentacion.md: relevamiento del requerimiento y decisiones.
- inventario_y_hallazgos.md: inventario del repo del equipo y comparacion con el requerimiento.
- db/01_schema.sql, db/02_trigger.sql, db/03_pruebas.sql: esquema, trigger y casos de prueba.
- DESIGN.md / INFRA.md: lineamientos de diseno y de infraestructura/despliegue.
- Este documento (defensa por criterios) y el diagrama ER en Azimutt.

Anexo: preguntas de diseno (defensa)

1. Para que sirve la tabla MONEDAS?

No es decorativa: la usa el trigger. El importe de cada cuota se calcula con `TRUNCATE(total / cuotas, MONEDAS.decimales)`. Ese campo 'decimales' (Guarani=0, Dolar=2) define cuanto se redondea cada cuota; sin la tabla habria que hardcodearlo y se romperia al facturar en otra moneda. Ademias el modelo del cliente muestra 'Moneda: Guarani' en la cuenta y VENTAS/EMPRESAS la referencian. Justificacion: el calculo de la cuota depende de los decimales de la moneda.

2. Faltan relaciones? (PRODUCTO_DETALLE y CUENTAS_COBRAR)

PRODUCTO_DETALLE con VENTA_DETALLES: si faltaba una clave foranea. La relacion correcta es `VENTA_DETALLES.codbarra -> PRODUCTO_DETALLE.codbarra` (una linea de venta apunta a un producto, no al reves). Se agrego la FK 'fkvtadetprod' al esquema y a la base.

CUENTAS_COBRAR con PLAZO: no hace falta y es a proposito. El plazo ya esta en `VENTAS.plazoid`; la cuota se alcanza por `CUENTAS_COBRAR -> VENTAS -> PLAZOS`. Una vez generada, la cuota es un hecho consolidado (importe + vence): el plazo ya hizo su trabajo. Atarla a PLAZO seria redundante y ademias rompe la idea de que CUENTAS_COBRAR sea reusable para compras.

3. CUENTAS_COBRAR no tiene relacion fisica con otras tablas?

Correcto: no tiene clave foranea fisica. Su relacion es logica, via las columnas (tabla, tabloid) que apuntan a VENTAS (tabla='VENTAS', tabloid=VENTAS.id). Es una relacion polimorfica: el mismo modelo de cuentas sirve para ventas (cobrar) y, a futuro, compras (pagar), sin duplicar la tabla. El trade-off es que la integridad de esa relacion no la garantiza la base (no hay FK), sino la logica de la aplicacion y del trigger. La alternativa seria poner una FK directa a VENTAS, pero entonces no podria apuntar tambien a COMPRAS; se eligio la flexibilidad a proposito.

4. El cliente no pide modulo de compras?

Si lo menciona ('los procesos de compras y ventas'), pero el entregable lo acota: el punto 3 pide ensayar el trigger 'para cualquiera de los procesos (compras o ventas)'. Se implemento ventas, que es valido. Compras es el proceso espejo (`COMPRAS + COMPRA_DETALLES + cuentas a pagar`) y se resuelve reusando CUENTAS_COBRAR con `tabla='COMPRAS'`: es una extension chica, no un rediseño.